

◊ ● BASIC LEVEL (1-15)

1 ☐ Reverse string (no slicing)

```
def reverse_string(s):  
    res = ""  
    for ch in s:  
        res = ch + res  
    return res
```

2 ☐ Palindrome check

```
def is_palindrome(s):  
    l, r = 0, len(s)-1  
    while l < r:  
        if s[l] != s[r]:  
            return False  
        l += 1  
        r -= 1  
    return True
```

3 ☐ Count vowels & consonants

```
def count_vc(s):  
    v = c = 0  
    vowels = "aeiouAEIOU"  
    for ch in s:  
        if ch.isalpha():  
            if ch in vowels:  
                v += 1  
            else:  
                c += 1  
    return v, c
```

4 ☐ Count upper, lower, digit, special

```
def count_types(s):  
    u=l=d=sp=0  
    for ch in s:  
        if ch.isupper(): u+=1  
        elif ch.islower(): l+=1  
        elif ch.isdigit(): d+=1  
        else: sp+=1  
    return u,l,d,sp
```

5 ☐ Remove duplicates

```
def remove_duplicates(s):  
    res=""  
    for ch in s:  
        if ch not in res:  
            res+=ch  
    return res
```

6 ☐ Frequency of characters

```
def frequency(s):
```

```

freq={}
for ch in s:
    freq[ch]=freq.get(ch,0)+1
return freq

```

7 ☐ First non-repeating char

```

def first_non_repeat(s):
    freq=frequency(s)
    for ch in s:
        if freq[ch]==1:
            return ch
    return None

```

8 ☐ First repeating char

```

def first_repeat(s):
    seen=set()
    for ch in s:
        if ch in seen:
            return ch
        seen.add(ch)

```

9 ☐ Only digits (no isdigit)

```

def only_digits(s):
    for ch in s:
        if ch<'0' or ch>'9':
            return False
    return True

```

10 ☐ Toggle case

```

def toggle_case(s):
    res=""
    for ch in s:
        if 'a'<=ch<='z':
            res+=chr(ord(ch)-32)
        elif 'A'<=ch<='Z':
            res+=chr(ord(ch)+32)
        else:
            res+=ch
    return res

```

11 ☐ Count words (no split)

```

def count_words(s):
    count=0
    in_word=False
    for ch in s:
        if ch!=" " and not in_word:
            count+=1
            in_word=True
        elif ch==" ":
            in_word=False

```

```
return count
```

12 Largest word

```
def largest_word(s):
    words=s.split()
    largest=""
    for w in words:
        if len(w)>len(largest):
            largest=w
    return largest
```

13 Remove spaces

```
def remove_spaces(s):
    res=""
    for ch in s:
        if ch!=" ":
            res+=ch
    return res
```

14 Replace vowels with *

```
def replace_vowels(s):
    vowels="aeiouAEIOU"
    res=""
    for ch in s:
        if ch in vowels:
            res+="*"
        else:
            res+=ch
    return res
```

15 Count substring occurrence

```
def count_substring(s, sub):
    count=0
    for i in range(len(s)-len(sub)+1):
        if s[i:i+len(sub)]==sub:
            count+=1
    return count
```

◊ ● INTERMEDIATE (16-30)

16 Anagram (no sorted)

```
def is_anagram(a,b):
    if len(a)!=len(b): return False
    freq={}
    for ch in a:
        freq[ch]=freq.get(ch,0)+1
    for ch in b:
        if ch not in freq: return False
        freq[ch]-=1
    return True
```

17 ☐ Rotation check

```
def is_rotation(a,b):  
    return len(a)==len(b) and b in a+a
```

18 ☐ All substrings

```
def all_substrings(s):  
    res=[]  
    for i in range(len(s)):  
        for j in range(i+1,len(s)+1):  
            res.append(s[i:j])  
    return res
```

19 ☐ Permutations (Recursion)

```
def permutations(s):  
    if len(s)==1:  
        return [s]  
    res=[]  
    for i,ch in enumerate(s):  
        for p in permutations(s[:i]+s[i+1:]):  
            res.append(ch+p)  
    return res
```

20 ☐ Second most frequent

```
def second_most(s):  
    freq=frequency(s)  
    values=sorted(freq.values(),reverse=True)  
    second=values[1]  
    for k,v in freq.items():  
        if v==second:  
            return k
```

21 ☐ Remove consecutive duplicates

```
def remove_consecutive(s):  
    res=s[0]  
    for i in range(1,len(s)):  
        if s[i]!=s[i-1]:  
            res+=s[i]  
    return res
```

22 ☐ String compression

```
def compress(s):  
    res=""  
    count=1  
    for i in range(1,len(s)):  
        if s[i]==s[i-1]:  
            count+=1  
        else:  
            res+=s[i-1]+str(count)  
            count=1  
    res+=s[-1]+str(count)
```

```
    return res
```

23 ☐ Valid palindrome ignoring special

```
def valid_palindrome(s):
    filtered=""
    for ch in s:
        if ch.isalnum():
            filtered+=ch.lower()
    return filtered==filtered[::-1]
```

24 ☐ Longest substring without repeat

```
def longest_unique(s):
    seen=set()
    l=0
    max_len=0
    for r in range(len(s)):
        while s[r] in seen:
            seen.remove(s[l])
            l+=1
        seen.add(s[r])
        max_len=max(max_len,r-l+1)
    return max_len
```

25 ☐ Duplicate characters with frequency

```
def duplicate_freq(s):
    freq=frequency(s)
    return {k:v for k,v in freq.items() if v>1}
```

26 ☐ Custom split

```
def custom_split(s):
    words=[]
    word=""
    for ch in s:
        if ch==" ":
            if word:
                words.append(word)
                word=""
            else:
                word+=ch
        if word:
            words.append(word)
    return words
```

27 ☐ Custom replace

```
def custom_replace(s,old,new):
    return s.replace(old,new)
```

28 ☐ Reverse words

```
def reverse_words(s):
    words=s.split()
```

```
return " ".join(words[::-1])
```

29 ☐ Isomorphic strings

```
def isomorphic(s,t):
    map1,map2={},{}
    for a,b in zip(s,t):
        if map1.get(a,b)!=b or map2.get(b,a)!=a:
            return False
        map1[a]=b
        map2[b]=a
    return True
```

30 ☐ Common characters

```
def common_chars(a,b):
    return list(set(a)&set(b))
```

31 ☐ Longest substring without repeating
try yourself

32 ☐ At most K distinct

```
def k_distinct(s,k):
    l=0
    freq={}
    max_len=0
    for r in range(len(s)):
        freq[s[r]]=freq.get(s[r],0)+1
        while len(freq)>k:
            freq[s[l]]-=1
            if freq[s[l]]==0:
                del freq[s[l]]
            l+=1
        max_len=max(max_len,r-l+1)
    return max_len
```

33 ☐ Minimum window substring
try yourself

34 ☐ Group anagrams

```
def group_anagrams(strs):
    groups={}
    for word in strs:
        key="".join(sorted(word))
        groups.setdefault(key,[]).append(word)
    return list(groups.values())
```

35 ☐ Valid parentheses

```
def valid_parentheses(s):
    stack=[]
    pairs={'(':')','{':'}','[':']'}
    for ch in s:
```

```

        if ch in "({[":
            stack.append(ch)
        else:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack

```

36 ☐ Longest common prefix

```

def lcp(strs):
    prefix=strs[0]
    for word in strs[1:]:
        while not word.startswith(prefix):
            prefix=prefix[:-1]
    return prefix

```

37 ☐ Longest palindromic substring
try yourself

38 ☐ Roman to integer

```

def roman_to_int(s):
    roman={'I':1, 'V':5, 'X':10, 'L':50, 'C':100, 'D':500, 'M':1000}
    total=0
    prev=0
    for ch in reversed(s):
        val=roman[ch]
        if val<prev:
            total-=val
        else:
            total+=val
        prev=val
    return total

```

39 ☐ Implement atoi

```

def my_atoi(s):
    s=s.strip()
    sign=1
    i=0
    if s[0]=='-':
        sign=-1
        i+=1
    num=0
    while i<len(s) and s[i].isdigit():
        num=num*10+ord(s[i])-ord('0')
        i+=1
    return sign*num

```

40 ☐ Find all anagram indices

```

def find_anagrams(s,p):
    from collections import Counter
    res=[]

```

```

p_count=Counter(p)
window=Counter()
for i in range(len(s)):
    window[s[i]]+=1
    if i>=len(p):
        if window[s[i-len(p)]]==1:
            del window[s[i-len(p)]]
        else:
            window[s[i-len(p)]]-=1
    if window==p_count:
        res.append(i-len(p)+1)
return res

```

41 KMP Algorithm (Pattern Matching)

Step 1: Build LPS (Longest Prefix Suffix)

```

def build_lps(pattern):
    lps = [0] * len(pattern)
    length = 0
    i = 1

    while i < len(pattern):
        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
            i += 1
        else:
            if length != 0:
                length = lps[length - 1]
            else:
                lps[i] = 0
                i += 1
    return lps

```

Step 2: KMP Search

```

def kmp_search(text, pattern):
    lps = build_lps(pattern)
    i = j = 0
    result = []

    while i < len(text):
        if text[i] == pattern[j]:
            i += 1
            j += 1

        if j == len(pattern):
            result.append(i - j)
            j = lps[j - 1]

        elif i < len(text) and text[i] != pattern[j]:

```



```

        if j != 0:
            j = lps[j - 1]
        else:
            i += 1

    return result

```

● 42 Rabin-Karp Algorithm

```

def rabin_karp(text, pattern):
    n, m = len(text), len(pattern)
    base = 256
    prime = 101

    hash_p = 0
    hash_t = 0
    h = 1

    for i in range(m-1):
        h = (h * base) % prime

    for i in range(m):
        hash_p = (base * hash_p + ord(pattern[i])) % prime
        hash_t = (base * hash_t + ord(text[i])) % prime

    result = []

    for i in range(n - m + 1):
        if hash_p == hash_t:
            if text[i:i+m] == pattern:
                result.append(i)

        if i < n - m:
            hash_t = (base*(hash_t - ord(text[i])*h) + ord(text[i+m])) % prime
            if hash_t < 0:
                hash_t += prime

    return result

```

● 43 Z-Algorithm

```

def z_algorithm(s):
    n = len(s)
    Z = [0]*n
    l = r = 0

    for i in range(1, n):
        if i <= r:
            Z[i] = min(r-i+1, Z[i-l])
        while i+Z[i] < n and s[Z[i]] == s[i+Z[i]]:
            Z[i] += 1
        if i+Z[i]-1 > r:

```

```

        l, r = i, i+Z[i]-1
    return Z

```

Pattern search using Z:

```

def z_search(text, pattern):
    concat = pattern + "$" + text
    Z = z_algorithm(concat)
    m = len(pattern)
    result = []
    for i in range(len(Z)):
        if Z[i] == m:
            result.append(i - m - 1)
    return result

```

● 44 Edit Distance (Levenshtein Distance)

```

def edit_distance(a, b):
    m, n = len(a), len(b)
    dp = [[0]*(n+1) for _ in range(m+1)]

    for i in range(m+1):
        dp[i][0] = i
    for j in range(n+1):
        dp[0][j] = j

    for i in range(1, m+1):
        for j in range(1, n+1):
            if a[i-1] == b[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j],      # delete
                                   dp[i][j-1],      # insert
                                   dp[i-1][j-1])      # replace

    return dp[m][n]

```

● 45 Word Break Problem (DP)

```

def word_break(s, wordDict):
    word_set = set(wordDict)
    dp = [False]*(len(s)+1)
    dp[0] = True

    for i in range(1, len(s)+1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True
                break
    return dp[len(s)]

```

● 46 Count Palindromic Substrings

```

def count_palindromes(s):
    count = 0

    for center in range(len(s)):
        l = r = center
        while l >= 0 and r < len(s) and s[l] == s[r]:
            count += 1
            l -= 1
            r += 1

        l, r = center, center+1
        while l >= 0 and r < len(s) and s[l] == s[r]:
            count += 1
            l -= 1
            r += 1

    return count

```

47 Partition String into Palindromes

```

def partition(s):
    result = []

    def backtrack(start, path):
        if start == len(s):
            result.append(path[:])
            return
        for end in range(start+1, len(s)+1):
            sub = s[start:end]
            if sub == sub[::-1]:
                backtrack(end, path+[sub])

    backtrack(0, [])
    return result

```

48 Regular Expression Matching (DP)

Supports . and *

```

def is_match(s, p):
    dp = [[False]*(len(p)+1) for _ in range(len(s)+1)]
    dp[0][0] = True

    for j in range(2, len(p)+1):
        if p[j-1] == "*":
            dp[0][j] = dp[0][j-2]

    for i in range(1, len(s)+1):
        for j in range(1, len(p)+1):
            if p[j-1] == "." or p[j-1] == s[i-1]:
                dp[i][j] = dp[i-1][j-1]

```

```

        elif p[j-1] == "*":
            dp[i][j] = dp[i][j-2]
            if p[j-2] == "." or p[j-2] == s[i-1]:
                dp[i][j] |= dp[i-1][j]
    return dp[len(s)][len(p)]

```

49 Text Justification

```

def full_justify(words, maxWidth):
    res = []
    i = 0

    while i < len(words):
        line_len = len(words[i])
        j = i + 1

        while j < len(words) and line_len + len(words[j]) + (j-i) <= maxWidth:
            line_len += len(words[j])
            j += 1

        spaces = maxWidth - line_len
        gaps = j - i - 1

        if j == len(words) or gaps == 0:
            line = " ".join(words[i:j])
            line += " "*(maxWidth - len(line))
        else:
            space, extra = divmod(spaces, gaps)
            line = ""
            for k in range(i, j-1):
                line += words[k] + " "*(space + (1 if k-i < extra else 0))
            line += words[j-1]

        res.append(line)
        i = j

    return res

```

50 Suffix Array Construction

```

def build_suffix_array(s):
    suffixes = [(s[i:], i) for i in range(len(s))]
    suffixes.sort()
    return [suffix[1] for suffix in suffixes]

```